

claude-windows / 068560d3-1ca2-4387-981a-d2d6bca55414

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\068560d3-1ca2-4387-981a-d2d6bca55414\subagents\workflows\wf_b3a30c42-346\agent-a603151fb7d352352.jsonl`  
- SHA-256: `4075f253cc0474703478791e0e82328d7e4118d543d535918f1a6546bf4879b5`  
- Source modified: `2026-06-16T02:27:36+00:00`  
- Imported at: `2026-07-05T16:48:28+00:00`  
- project: `wf_b3a30c42-346`  
- session_id: `068560d3-1ca2-4387-981a-d2d6bca55414`
```

Transcript

1. user (2026-06-16T02:24:52.620Z)

Determine the repo's constraints for committing structured regression fixtures. Read docs/CODE_MAP.md section 0 (file-placement table); read .gitignore; check for git-lfs (.gitattributes); read the DB migration-ladder conventions in backend/infrastructure/database.py (PRAGMA user_version) and any schema-version patterns; review licensing/privacy guardrails in CLAUDE.md and docs (MIT/Apache/BSD only; paid LLMs never a training source; exclude minors; per-user opt-in). ANSWER: (1) Where do committed test fixtures belong per file-placement rules, and what is gitignored (artifacts/, scratch/)? (2) Is git-lfs in use or available? (3) What schema/version conventions exist that fixtures should mirror? (4) LICENSING/PRIVACY: is committing detector-DERIVED data (shuttle coordinates, segment intervals, audio-onset features) from labeled videos to the GitHub repo acceptable given the guardrails and the likely source of the golden videos (collector/self-recorded vs third-party broadcast)? What must the design do to stay clean (commit only derived numeric streams not frames? only owned/cleared videos? exclude minors?)? (5) Rough size budget: expected number of golden videos and size of a per-video trajectory stream. Flag concerns.

2. assistant (2026-06-16T02:24:56.349Z)

I'll investigate the repo's constraints for committing structured regression fixtures. Let me start by reading the key files in parallel.

3. user (2026-06-16T02:24:59.888Z)

fatal: Cannot fast-forward to multiple branches.

4. user (2026-06-16T02:24:59.911Z)

```
1      # Byte-compiled / optimized / DLL files  
2      __pycache__/  
3      *.py[cod]  
4      *$py.class  
5      *.pt  
6  
7      # Local environment secrets  
8      .env  
9      *.env  
10     .env.*  
11  
12     # C extensions  
13     *.so  
14  
15     # Distribution / packaging  
16     .Python  
17     build/  
18     develop-eggs/
```

```

19     dist/
20     downloads/
21     eggs/
22     .eggs/
23     lib/
24     lib64/
25     parts/
26     sdist/
27     var/
28     wheels/
29     share/python-wheels/
30     *.egg-info/
31     .installed.cfg
32     *.egg
33     MANIFEST
34
35     # SQLite databases & Local output files
36     *.db
37     *.db-journal
38     *.db-wal
39     output/
40     output_dir/
41     backend/static/highlights_*.mp4
42     backend/static/*_compiled.mp4
43
44     # OS generated files
45     .DS_Store
46     Thumbs.db
47     desktop.ini
48
49     # Coverage data
50     .coverage
51     .coverage.*
52     htmlcov/
53     .pytest_cache/
54
55     # Local Claude Code config (machine-specific)
56     .claude/
57     .clocovleerlaage
58     ■
59     ■
60     # Throwaway repro scripts + PR-body / handoff scratch notes (already excluded from
61     # ruff + pytest; never meant to be committed ? keep them out of `git status` too).
62     scratch/
63     .venv/
64

```

5. user (2026-06-16T02:25:00.005Z)

No files found

6. assistant (2026-06-16T02:25:03.310Z)

The pull failed because of multiple upstream branches configured. Let me note that and continue reading the key files in parallel.

7. user (2026-06-16T02:25:06.293Z)

```

git-lfs/3.5.1 (GitHub; windows amd64; go 1.21.7; git e237bb3a)
---LFS-INSTALLED-CHECK---
filter.lfs.clean git-lfs clean -- %f
filter.lfs.smudge git-lfs smudge -- %f
filter.lfs.process git-lfs filter-process
filter.lfs.required true

```

---ATTRIBUTES---

ls: cannot access '.gitattributes': No such file or directory

---BRANCH---

master

b6a39c4 feat(runner): fully-local no-AI segmenter mode (+ WASB stream flag, .env load) (#183)

8. user (2026-06-16T02:25:06.562Z)

```
1      # CODE_MAP.md
2
3      **Purpose:** cold-start navigation map for the badminton-highlight-indexer ? a self-
hosted video ? highlight-reel pipeline (CV/AI rally detection ? SQLite ? ffmpeg stitch) with a
segmenter-agnostic eval/calibration stack, typed config, and per-video observability reporting.
4
5      **Regenerate:** rerun the `build-code-map` workflow (parallel subsystem readers ? this
doc). Re-run after any registry/contract/WSL-interface change, or after a directory restructure
(this revision follows #40).
6
7      ? **2026-06-10 audit-hardening sweep changed several behaviors documented below ? trust
git history over stale lines until the next full regen:** `add_video` is now an **UPSERT** (no
FK-cascade wipe on re-ingest; segmentation is destroy-AFTER-confirm via
`segment_watermarks`/`prune_segments`); detector artifacts are keyed by **video_id**, not
basename; **`indexing.wasb.*` is now a real typed config** (was silently dropped); `timeout_sec`
defaults **1800s** (was 0/hang-forever); SQLite opens with **WAL + busy_timeout**;
```

`match\_intervals` requires real overlap (iou>0); `run\_regression` baseline lives in tracked
`eval\_baselines/` and **exits 3 on a missing baseline**;

the API validates
`video\_path`/`output\_filename`. New tests: `test\_api\_endpoints`, `test\_compiler`,
`test\_wasb\_runner`, `test\_reingest\_safety`, `test\_detector\_keying`, `test\_config\_wasb`,
`test\_path\_safety`, `test\_reliability\_hardening` (+ collector `test\_licensing\_gate`,
`test\_import\_guard`, `test\_cvat\_robustness`).

```
8
9      **LEGEND**
10     - `@path` = file (always repo-relative)
11     - `:::sym` = symbol (class/func/const) in the nearest preceding `@path`
12     - `->` = dataflow / call / "produces"
13     - `$` = data contract (canonical shape)
14     - `?` = gotcha / footgun
15     - `?` = registry / plugin extension point
16     - `WSL` = runs inside WSL2 conda env, NOT the Windows Python process
17
18     ---
19
20     ## 0. REPO LAYOUT ? where new files go (READ BEFORE committing a new file)
21
22     Top-level tree and the **placement rule** for each kind of file. When unsure, prefer an
existing
23     **registry / extension point** (`?`) over a new top-level file.
24
25     | You're adding? | Put it in? | Notes |
26     |---|---|---|
27     | Backend / runtime code | `backend/<subsystem>/` |
`pipeline/{segmenters,detectors,validators}`, `providers/`, `sports/`, `annotations/`,
`infrastructure/`, `reporting/`, `stitcher/`, `utils/`, `config/`, `features/`, `api/`. Most
additions are **register a class in the relevant `?` registry**, not a new module tree. |
28     | **Eval LIBRARY** (importable, pure ? metrics, calibration, a scorer, a feature) |
`backend/eval/` | Pure core, no side-effects at import; CLI/I-O lives in a separate driver.
These are **imported widely** (e.g. `calibrate_wasb` by 8 sites) ? they are library, not
scripts; do NOT move them out. |
29     | **Standalone run-driver / one-off entry CLI** (has `__main__`, NOT imported by app
code) | **`scripts/`** | e.g. `scripts/run_phase3_eval.py`, `scripts/run_process_and_stitch.py`.
Run as `python scripts/<name>.py`. If it needs `load_dotenv()`/`sys.path` before importing
backend, add `scripts/<name>.py" = ["E402"]` to `ruff.toml`. |
30     | **Ops / maintenance tool** (not part of the served app ? cache GC, batch admin) |
`tools/` | Run as `python -m tools.<name>` (it's an importable package). e.g.
`tools/cleanup_caches.py`. Keep the destructive decision path pure + unit-tested. |
```

```

31 | Training code (model fitting) | `training/` | Behind the CI-enforced import wall
(ADR-008) ? `backend.main` must not import it. Own `requirements-train.txt`. |
32 | A test | `tests/test_*.py` | pytest auto-discovers; the full-suite lane has a **70%
coverage floor**. Pure/offline + mocked (no GPU/WSL/network). |
33 | A doc (design, plan, living log) | `docs/` | Index it in `docs/README.md`. Only
**terminal-state** (DONE/REJECTED) work moves to `docs/archives/`. |
34 | A throwaway repro / debug script | `scratch/` | **Gitignored** + excluded from
ruff/pytest. Never imported by app code. |
35 | A model artifact (weights + manifest) | `artifacts/<name>/<ver>/` | ADR-008:
**manifest committed** (provenance), **weights gitignored**. |
36 | Eval baseline / leaderboard JSON | `eval_baselines/` | Tracked (survives a clean
checkout); the no-regression gate reads it. |
37 | Pipeline run outputs (reels, trajectories, DBs, frames) | `output/` | **Gitignored.**
Per-video isolation is planned in `PER_VIDEO_OUTPUT_LAYOUT.md`. |
38
39 **Canonical files that stay at the repo ROOT** (referenced by CI / packaging / docs ? do
not move):
40 `pyproject.toml` (PEP 621 packaging, hatchling; `indexer` entry point) .
`run_all_tests.py` (CI: `ci.yml`) .
41 `run_regression.py` (the no-regression gate CLI, cited in `EXPERIMENT_HARNESS.md`) .
`run_eval.py`
42 (grandfathered ? imported by `tests/test_eval_harness.py`). The old diagnostic
`setup.py` moved to
43 `scripts/doctor.py` (it shadowed the build system); invoke via `python
scripts/doctor.py` or `indexer --setup`.
44 New standalone run-drivers go in **`scripts/`**, not the root.
45
46 ---
47
48 ## 1. PIPELINE
49
50 ### 1A. Runtime: video file -> highlight reel
51 ```
52 typed config load (built-in defaults -> config.json overrides; absent/parse-fail -> all-
defaults, never fatal)
53                                     @backend/config/loader.py::load_config ->
@backend/config/models.py::AppConfig
54 -> video_id = MD5(whole file)                                     @backend/utils/hashing.py::compute_video_id
(single shared helper, 64KB chunks; imported by main + all run_*.py ? no more per-entrypoint
copies)
55 -> validate
@backend/pipeline/validators/base.py::registry.run_all_with_context(video_path, global_config)
56 FormatValidator -> ctx['ffprobe_meta'] -> ResolutionFPSValidator -> PTSContinuity
-> SceneCut -> Blur -> Relevance (? ACTUAL registration/exec order ? see
@backend/pipeline/validators/__init__.py)
57 [validation FAIL -> persist status="failed", return early; report STILL emitted
in finally]
58 -> db.add_video(video_id, filepath, status, [r.dict()])
@backend/infrastructure/database.py
59 -> seg_name = req.segmenter or global_config.indexing.default_segmenter (fallback
'motion')
60 -> seg = segmenter_registry.get(seg_name)(db, global_config) ?
@backend/pipeline/segmenters/base.py::segmenter_registry (400 + available list if missing)
61 -> segments = seg.process_video(video_id, video_path, player_pool?, max_frames?)
62 [STOP POINT: segments-only ? predictions now in DB; eval/regression operate here
without stitching]
63 -> finally: emit_indexer_report(...) @backend/reporting/__init__.py (? ALWAYS runs
? even on validation failure / exception; NEVER raises; grafts response["reports"])
64 -> select segment ids -> [(start,end)] (+ stitching padding)
65 -> VideoCompiler.compile_highlights(filepath, time_pairs, out_name)
@backend/pipeline/stitcher/compiler.py
66 per-clip ffmpeg re-encode (libx264/superfast/crf22) -> concat -c copy ->
output/<name>.mp4
67 ```
68 Hybrid segmenter internal 4-phase (yolo_hybrid / tracknet_hybrid / wasb_hybrid ? same

```

```

shape):
69     ...
70     P1 chunk video (yolo only; tracknet/wasb = whole-video single pass, chunk_index=0)
71     -> P2 candidate "action windows" from motion/trajectory [STOP: candidates persisted
via db.add_candidate_segment]
72     -> P3 pad clip (ai_handoff_padding) -> ThreadPoolExecutor ->
provider.analyze_video(clip, prompt) ? provider_registry
73     -> P4 db.add_play_segment + db.link_candidate_to_segment
74     ...
75     Pure-AI path `gemini`: no P2; chunk-or-single -> provider -> heavy validate
(clamp/dedup) -> add_play_segment.
76     Legacy `motion`: pixel-diff -> segments + **fabricated** metadata/events (? not ground
truth).
77
78     ### 1B. WASB shuttle substrate (P2 of wasb_hybrid)
79     ...
80     WasbHybridSegmenter.process_video
@backend/pipeline/segmenters/wasb_hybrid.py
81     -> WasbRunner.healthcheck() (gate; not ok -> return [])
@backend/pipeline/detectors/wasb_runner.py
82     -> WasbRunner.run_predict(video, out_dir)
83     stage video + wasb_infer.py into WSL fs -> `wsl bash -c 'source conda.sh && python
wasb_infer.py ...'`
84     -> @backend/pipeline/detectors/wasb_infer.py::run (WSL): sliding windows -> GPU
detector pass (CHECKPOINTED det_raw.jsonl) -> CPU tracker (always full re-run) -> trajectory CSV
85     -> TrackNetRunner.parse_trajectory_csv(csv)
@backend/pipeline/detectors/tracknet_runner.py (shared, re-exported by WasbRunner)
86     -> TrackNetRunner.trajectory_to_action_windows(points, fps, frame_width, chunk_start,
velocity_thresh, merge_gap, min_window_duration, chunk_index) [the model-agnostic WINDOWING
BRAIN]
87     ...
88     (`tracknet_hybrid` identical, swapping WasbRunner->TrackNetRunner; `_probe_fps_width`
fallback (30.0, 1280.0).)
89
90     ### 1C. Calibration / eval flow (NO pipeline run unless told)
91     ...
92     GT CSV -> annotations.load_annotations / calibrate_wasb.load_golden_gts ->
List[Interval]
93     DB predictions -> harness.get_predictions(db, video_id, stage) -> List[Interval] stage
? {candidates, final}
94     -> metrics.match_intervals (greedy temporal-IoU) -> metrics.score -> Score
95     -> harness.evaluate -> EvalReport -> report_to_dict
96     -> batch.aggregate (corpus micro/macro) @backend/eval/batch.py
97     -> regression.regress(_with_provider) vs baseline @backend/eval/regression.py
[CI gate]
98     Calibration: calibrate_wasb.run -> calibration.{select_best, improvement_report,
cross_validate} @backend/eval/calibration.py
99     Distill Gemini->local: calibrate_local (thresholds) OR distill_local (learned
classifier) @backend/eval/{calibrate_local,distill_local}.py
100     Learned segmenter (offline): training.build_training_set(X,y) ->
classifier.LogisticRegression.fit/save
101     Gemini refinement driver (tuning, standalone):
gemini_refine.{refine_window,full_video_rallies} @backend/eval/gemini_refine.py
102     Audio probe (diagnostic only, NOT in live pipeline): audio/e1_probe.run
@backend/eval/audio/e1_probe.py
103     ...
104     Entrypoints: `@run_eval.py` (single video score), `@scripts/run_phase3_eval.py`
(manifest batch, can segment), `@run_regression.py` (baseline gate, exit 1=regression),
`@backend/eval/calibrate_wasb.py` + `@backend/eval/export_wasb_segments.py` (WASB tuning
drivers).
105
106     ---
107
108     ## 2. SUBSYSTEMS
109

```

```

110     ### 2.0 Orchestration & config
111     - `@backend/main.py` ? Thin CLI + FastAPI bootstrap (orchestrator; modes via FLAGS).
Static UI mounts `/static` from `backend/api/static`. Logging is initialized with `basicConfig`
(INFO, timestamped format); backend modules use `logger = logging.getLogger(__name__)` (replaces
stray prints in hot paths like motion, stitcher, wasb_infer, config loader). User-facing CLI
summaries may still use `print` for direct output.
112     - `::app` FastAPI; `::db_instance`, `::global_config` (`Optional[AppConfig]`) module
globals set ONLY in `main()`. ? importing `app` for ASGI without `main()` -> both `None` ->
every endpoint NPEs.
113     - **ASYNC (#16, PR-1):** `::process_video` `@POST /api/process` -> fast-fail 4xx
(path/exists/segmenter) -> `db.create_job` -> `_get_executor().submit(_run_job,...)` -> **202 +
job_id**. `::run_job` (worker: mark running -> `_execute_processing` -> done/failed; catches +
logs traceback). `::execute_processing` = the former sync body (hash -> validate -> add_video
-> segment -> `finally:` always `emit_indexer_report`, grafts `response["reports"]`; never
raises from the report); a RAISING segmenter now prune_after-restores pre-run rows before re-
raising. `::get_job` `@GET /api/jobs/{job_id}` -> `{status, progress, video_id, error, result,
reports, ...}` ? job `status` = EXECUTION state (`queued|running|done|failed`); the pipeline
verdict lives in `result["status"]`. `::get_executor` lazy module-global
`ThreadPoolExecutor(max_workers=1)` (single box + Gemini serial-upload learning; tests
monkeypatch it inline). `main()` runs `db.fail_stale_jobs()` at startup (orphaned queued/running
-> failed). `::compile_highlights` `@POST /api/compile`. `::query_play_segments` `@POST
/api/query`. `::get_config` `@GET /api/config`. `::run_cli_diagnose_clip` `--debug-clip`: lazy
`import cv2`, samples ±3s vs `indexing.motion_threshold` (dual model/dict read). ? CLI `--video-
path` mode still runs the pipeline synchronously inline (separate code path, unchanged by #16).
114     - **UPLOAD/DOWNLOAD (PR-2, B2+B3):** chunked upload ? `::upload_init` `@POST
/api/upload/init` (`{filename, total_size_bytes}` -> ext allowlist + size gate ->
`{upload_id}`), `::upload_part` `@PUT /api/upload/{id}/part/{n}` (raw-bytes body, STRICTLY
sequential n, streams to `.partial-<id>` under `security.upload_dir`; per-part/total cap breach
-> 413 + upload ABORTED), `::upload_complete` `@POST .../complete` (`{total_parts}` -> assemble
-> never-overwrite rename -> `{path, video_id(MD5), size_bytes}` ? client then POSTs
/api/process with that path), `::upload_abort` `@DELETE /api/upload/{id}`. ? upload state
(`::uploads` + `::uploads_lock`) is IN-PROCESS ? restart aborts partials (client re-inits);
per-user partitioning lands with PR-3. `::download_highlight` `@GET
/api/highlights/{video_id}/{filename}` ? safe-name + `resolve_under_root(output_dir)` then
FileResponse stream (the old /api/compile alert path was browser-unreachable). Config:
`SecurityConfig.{upload_dir='output/uploads', max_upload_mb=4096, max_part_mb=95,
allowed_upload_extensions=[mp4,mov]}` (95MB: free-tier edge proxies cap bodies ~100MB).
115     - `::load_config` ? ? legacy thin wrapper returning a **dict**
(`load_app_config(path).model_dump(...)`); kept for transition ? new code imports
`load_app_config`/`AppConfig` directly. `::ProcessRequest`/`::QueryRequest`/`::CompileRequest`
pydantic bodies.
116     - `main()` sets `global_config.output.output_dir = args.output_dir` (typed field on
`OutputConfig`); `db_path = os.path.join(global_config.output.output_dir, output.db_name)`.
`compile_highlights` reads the SAME `global_config.output.output_dir`, so `--output-dir` is
honored end-to-end (the old write-only `runtime_overrides` hack is gone).
117     - ? `--segmenter` argparse `choices` frozen at build time from
`segmenter_registry.list_available()` ? a lazily-registered segmenter wouldn't appear. CLI
default_segmenter fallback `motion`.
118     - `@scripts/run_process_and_stitch.py` ? one-shot demo; ? hardcoded paths, interactive
`input()` (blocks automation), skips validation. Config via `backend.config.load_config`
(typed); segmenter, padding, and db path all read from the model (no literal fallbacks).
119     - `@run_eval.py::main` ? score one video's DB preds vs GT CSV.
`--stage{candidates,final,both}`. Exit 2=arg/IO. ? pure scorer; errors if video never processed.
`::default_db_path` resolves via `backend.config.load_config`
(`output.output_dir`/`output.db_name`). `get_file_md5` is an alias for `compute_video_id`.
120     - `@scripts/run_phase3_eval.py::main` ? walk manifest, segment-if-needed, score,
aggregate. `load_dotenv()` at import. ? DB key = file MD5, NOT manifest's `video_id` (kept as
label); config via `backend.config.load_config`; `segmenter_cls(db, cfg)` receives the typed
`AppConfig` (same object the live pipeline feeds segmenters).

```

9. assistant (2026-06-16T02:25:10.755Z)

Key facts so far: git-lfs 3.5.1 is installed and configured globally (filters present) but there's NO `.gitattributes` in the repo, so LFS is not actually tracking anything. The file-placement table is clear. Now let me read the DB migration ladder and the licensing/privacy

docs.

10. user (2026-06-16T02:25:11.720Z)

```
1     import sqlite3
2     import json
3     import logging
4     from contextlib import contextmanager
5     from typing import List, Dict, Any, Optional, Tuple
6
7     logger = logging.getLogger(__name__)
8
9     class Database:
10         def __init__(self, db_path: str):
11             self.db_path = db_path
12             self._init_db()
13
14         @contextmanager
15         def _connect(self):
16             """Transaction scope that also CLOSES the connection.
17
18             ``with sqlite3.connect(...)`` alone is only a commit/rollback scope ? it
19             never closes, so every call leaked its connection to GC (lingering file
20             handles on the WAL sidecars under Windows). Internal methods use this.
21             """
22             conn = self._get_connection()
23             try:
24                 with conn:
25                     yield conn
26             finally:
27                 conn.close()
28
29         def _get_connection(self):
30             # busy_timeout: wait (not fail) on a locked DB ? needed once the async job model
31             # adds concurrent writers (a swallowed 'database is locked' silently drops
32             segments).
33             conn = sqlite3.connect(self.db_path, timeout=30.0)
34             conn.row_factory = sqlite3.Row
35             # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
36             conn.execute("PRAGMA foreign_keys = ON")
37             # WAL allows concurrent readers during a write; busy_timeout bounds lock waits.
38             conn.execute("PRAGMA busy_timeout = 30000")
39             try:
40                 conn.execute("PRAGMA journal_mode = WAL")
41             except sqlite3.OperationalError:
42                 pass # e.g. a read-only/networked FS that can't do WAL ? degrade gracefully
43             return conn
44
45         def _init_db(self):
46             """Initializes tables for videos, play_segments, and events."""
47             with self._connect() as conn:
48                 cursor = conn.cursor()
49
50                 # Videos Table
51                 cursor.execute("""
52                     CREATE TABLE IF NOT EXISTS videos (
53                         id TEXT PRIMARY KEY,
54                         filepath TEXT NOT NULL,
55                         processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
56                         status TEXT NOT NULL,
57                         validation_results TEXT
58                     )
59                 """)
60
61                 # Play Segments Table (Extensible metadata JSON)
```

```

61         cursor.execute("""
62             CREATE TABLE IF NOT EXISTS play_segments (
63                 id INTEGER PRIMARY KEY AUTOINCREMENT,
64                 video_id TEXT NOT NULL,
65                 start_time REAL NOT NULL,
66                 end_time REAL NOT NULL,
67                 duration REAL NOT NULL,
68                 server TEXT,
69                 receiver TEXT,
70                 metadata TEXT,
71                 FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
72             )
73         """)
74
75     # Events Table
76     cursor.execute("""
77         CREATE TABLE IF NOT EXISTS events (
78             id INTEGER PRIMARY KEY AUTOINCREMENT,
79             segment_id INTEGER NOT NULL,
80             timestamp REAL NOT NULL,
81             type TEXT NOT NULL,
82             player TEXT,
83             details TEXT,
84             FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE
85         )
86     """)
87
88     # Versioned migrations live here. PRAGMA user_version is the schema
89     # contract ? bump it for every change and add an ordered `if version < N`
90     # block below. Tests assert the expected version, so accidental drift fails
91     CI.
92     version = cursor.execute("PRAGMA user_version").fetchone()[0]
93
94     if version < 1:
95         # v1: raw candidate detections (pre-confirmation), detector-agnostic.
96         # Common fields are columns; detector-specific metrics go in `stats`
97         JSON,
98         # so a new segmenter reuses this table by setting its own
99         `detector`/`stats`.
100         cursor.execute("""
101             CREATE TABLE IF NOT EXISTS candidate_segments (
102                 id INTEGER PRIMARY KEY AUTOINCREMENT,
103                 video_id TEXT NOT NULL,
104                 detector TEXT NOT NULL,
105                 chunk_index INTEGER,
106                 start_time REAL NOT NULL,
107                 end_time REAL NOT NULL,
108                 duration REAL NOT NULL,
109                 confidence REAL,
110                 stats TEXT,
111                 detection_params TEXT,
112                 confirmed_segment_id INTEGER,
113                 human_label TEXT,
114                 notes TEXT,
115                 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
116                 FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
117                 FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id)
118             )
119         """)
120         cursor.execute("""
121             CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
122             ON candidate_segments(video_id, detector)
123         """)

```



```

121         cursor.execute("PRAGMA user_version = 1")
122
123     if version < 2:
124         # v2: async job model (DEFERRED_HARDENING_PLAN #16). One row per
submitted
125         # /api/process request. `status` is the EXECUTION state of the job
126         # (queued|running|done|failed); the pipeline's own verdict (e.g. a video
127         # that failed validation) lives inside `result` JSON as
result["status"].
128         # `result` is the full sync-era response dict (incl. report paths), so
the
129         # observability report is the job's artifact, per the plan.
130         cursor.execute("""

```

11. user (2026-06-16T02:25:12.278Z)

```

eval_baselines\heuristic_lovo_n6.json
eval_baselines\gemini_wrapper_2026-06-10.json
eval_baselines\experiment_leaderboard.json

```

12. user (2026-06-16T02:25:12.468Z)

No files found

13. assistant (2026-06-16T02:25:15.315Z)

Let me see the rest of the migration ladder, the eval_baselines JSON shape (these are the closest existing "committed fixtures"), and where existing test data/golden CSVs live.

14. user (2026-06-16T02:25:16.751Z)

```

130         cursor.execute("""
131             CREATE TABLE IF NOT EXISTS jobs (
132                 id TEXT PRIMARY KEY,
133                 video_path TEXT NOT NULL,
134                 video_id TEXT,
135                 segmenter TEXT,
136                 player_pool TEXT,
137                 max_frames INTEGER,
138                 status TEXT NOT NULL DEFAULT 'queued',
139                 progress TEXT,
140                 error TEXT,
141                 result TEXT,
142                 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
143                 started_at TIMESTAMP,
144                 finished_at TIMESTAMP
145             )
146         """)
147         cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_status ON
jobs(status)")
148         cursor.execute("PRAGMA user_version = 2")
149
150     if version < 3:
151         # v3: self-scheduling execution policy (EXECUTION_POLICY.md P2).
`policy` = the
152         # JSON ExecutionPolicy a job carries; `next_poll_at` = when a 'waiting'
job is due
153         # to resume. A 'waiting' status + due-time lets a job SELF-SCHEDULE ?
any worker
154         # re-claims it via `claim_due_job` when due ? with NO master node: the
table IS
155         # the coordinator. ALTER (not recreate) so existing v2 job rows survive.
156         cursor.execute("ALTER TABLE jobs ADD COLUMN policy TEXT")
157         cursor.execute("ALTER TABLE jobs ADD COLUMN next_poll_at TIMESTAMP")

```

```

158         cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_due ON jobs(status,
next_poll_at)")
159         cursor.execute("PRAGMA user_version = 3")
160
161     if version < 4:
162         # v4: personalization groundwork (PERSONALIZATION_PLAN.md §2). Add a
NULLABLE
163         # `user_id` to videos + candidate_segments so a future per-user / multi-
tenant
164         # path can scope rows to an owner. **NULL = local install = byte-
identical to
165         # today** ? every existing row stays NULL and every existing query
166         # filters on user_id) is unaffected. ALTER (not recreate) so v1-v3 rows
survive.
167         # Additive ONLY: no served-behavior change rides on this slice.
168         cursor.execute("ALTER TABLE videos ADD COLUMN user_id TEXT")
169         cursor.execute("ALTER TABLE candidate_segments ADD COLUMN user_id TEXT")
170         # Partial indexes (WHERE user_id IS NOT NULL) keep the NULL=local fast
path free
171         # of index churn while making the eventual per-user lookups cheap.
172         cursor.execute(
173             "CREATE INDEX IF NOT EXISTS idx_videos_user ON videos(user_id) "
174             "WHERE user_id IS NOT NULL")
175         cursor.execute(
176             "CREATE INDEX IF NOT EXISTS idx_candidates_user "
177             "ON candidate_segments(user_id) WHERE user_id IS NOT NULL")
178         cursor.execute("PRAGMA user_version = 4")
179
180     if version < 5:
181         # v5: per-user model store (PERSONALIZATION_PLAN.md §2). One row per
(user_id,
182         # version): the resolved per-user `fusion_config` overlay + an INLINE
evidence
183         # `classifier_json` (tenant-safe ? no file artifact), plus the promotion
184         # and provenance that earned it. `is_active` pins the one row the
serving bridge
185         # resolves (the FUTURE owner-gated wiring into `_select_for_handoff`).
This slice
186         # only persists/loads it ? NOTHING reads it on the served path yet.
187         #
188         #   user_id           ? owner of the model (NULL allowed = the local
install)
189         #   version           ? monotonic per-user model version
190         #   baseline_version   ? the shared baseline this was fit against
(upgrade switch)
191         #   feature_schema_hash ? fingerprint of the feature set; MINOR-vs-
MAJOR re-fit gate
192         #   fusion_config      ? JSON per-user FusionConfig overlay (cue_flags
/ thresholds)
193         #   classifier_json     ? JSON LogisticRegression.to_dict() (None =
config-only model)
194         #   promotion_evidence  ? JSON per_user_gate PromotionDecision (why it
was promoted)
195         #   n_videos_at_fit     ? held-out-user corpus size when fit (min_n
trust floor input)
196         #   is_active          ? the one resolved row for this user (partial-
unique below)
197         cursor.execute("""
198             CREATE TABLE IF NOT EXISTS user_models (
199                 id INTEGER PRIMARY KEY AUTOINCREMENT,
200                 user_id TEXT,
201                 version INTEGER NOT NULL DEFAULT 1,
202                 baseline_version TEXT,
203                 feature_schema_hash TEXT,

```

```

204         fusion_config TEXT,
205         classifier_json TEXT,
206         promotion_evidence TEXT,
207         n_videos_at_fit INTEGER NOT NULL DEFAULT 0,
208         is_active INTEGER NOT NULL DEFAULT 0,
209         created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
210     )
211     """)
212     # One model version per user (NULLs compare distinct in SQLite, so the
local
213     # install can still carry multiple versioned rows; the active-row guard
below
214     # is what enforces a single served model).
215     cursor.execute("""
216         CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_user_version
217         ON user_models(user_id, version)
218     """)
219     # At most ONE active model per user: a partial-unique index over the
active rows.

```

15. user (2026-06-16T02:25:17.701Z)

```

1      {
2      "name": "heuristic_velocity_windowing_LOVO",
3      "description": "The honest cross-video floor the Gen-0 learned model must beat:
calibrated velocity-windowing heuristic, leave-one-video-out over the n=6 owned golden corpus.",
4      "computed": "2026-06-10, reproduced via: python -m backend.eval.calibrate_local
--manifest output/human_lovo_manifest.json --metric f1 (matches docs/archives/quality-
iterations/2026-06-multivideo-lovo)",
5      "mean_f1": 0.480,
6      "std": 0.242,
7      "per_video": {
8      "adarsh_avi_singles": 0.751,
9      "kushagra_singles": 0.157,
10     "largetest_doubles": 0.737,
11     "gbaaddy": 0.309,
12     "testlarge_short": 0.267,
13     "mahadevpura_singles": 0.657
14     }
15     }
16

```

16. user (2026-06-16T02:25:19.703Z)

```

===CSV in repo===
===JSON fixtures in tests===
===tests data dirs===
tests/test_database.py
tests/test_fusion_golden_telemetry.py

```

17. assistant (2026-06-16T02:25:23.998Z)

No CSV files are committed anywhere in the repo, and `tests/` has no fixture data directories ? tests are fully synthetic. The only "committed structured data" precedent is `eval\_baselines/\*.json` (tiny aggregate metrics, no raw per-frame data). Let me check how tests generate their data and look at the golden-set/licensing docs and the existing fusion golden telemetry test.

18. user (2026-06-16T02:25:24.957Z)

```

1      """"GPU-free tests for the run-telemetry wire-in to the fusion_golden extractor.
2
3      `run()` builds a `RunTelemetry` black box in `out_dir` and emits video_start /
video_done events

```

```

4         around each extract_video; a hard kill during a long GPU run thus leaves a forensic
runlog. These
5     tests stub extract_video (no GPU) and inject deterministic collectors + clock via a
patched
6     RunTelemetry factory, so the JSONL content is asserted with no real nvidia-smi / psutil
/ wall-clock.
7     A telemetry-collector failure must NOT break extraction (graceful degradation).
8     """
9     import functools
10    import json
11    import os
12
13    from backend.eval import fusion_golden
14    from backend.utils import run_telemetry as rt
15
16
17    class _Clock:
18        def __init__(self, t=1000.0):
19            self.t = t
20
21        def __call__(self):
22            return self.t
23
24        def tick(self, dt):
25            self.t += dt
26
27
28    def _fake_gpu():
29        return {"temp_c": 70.0, "util_pct": 80.0, "throttled": False}
30
31
32    def _fake_sys():
33        return {"cpu_pct": 50.0, "ram_used_gb": 21.0}
34
35
36    def _patch_telemetry(monkeypatch, gpu_fn=_fake_gpu, sys_fn=_fake_sys):
37        """Make run()'s `RunTelemetry(...)` hermetic: inject deterministic collectors +
clock and a
38        fixed GPU-name header (no real nvidia-smi). Returns nothing; run() builds the
instance itself."""
39        monkeypatch.setattr(rt, "gpu_name", lambda: "FakeGPU")
40        factory = functools.partial(rt.RunTelemetry, gpu_fn=gpu_fn, sys_fn=sys_fn,
clock=_Clock())
41        monkeypatch.setattr(fusion_golden, "RunTelemetry", factory)
42
43
44    def _write_manifest(tmp_path, names):
45        manifest = [
46            {"name": n, "trajectory": f"{n}.csv", "fps": 30.0, "frame_width": 1920.0,
47             "labels": f"{n}.rallies.csv"}
48            for n in names
49        ]
50        mp = str(tmp_path / "m.json")
51        with open(mp, "w", encoding="utf-8") as f:
52            json.dump(manifest, f)
53        return mp
54
55
56    def _read_lines(path):
57        with open(path, encoding="utf-8") as f:
58            return [ln for ln in f.read().splitlines() if ln]
59
60

```

===git tracked under eval_baselines + any jsonl/csv anywhere===

backend/eval/fusion_golden.py
backend/pipeline/segmenters/trajectory_hybrid.py
docs/GOLDEN_DATA_SHARING.md
docs/GOLDEN_SET_IMPLEMENTATION.md
docs/GOLDEN_SET_PHASE1_VIDEOS.md
docs/GOLDEN_SET_VENDORING.md
tests/test_fusion_golden_telemetry.py
tests/test_trajectory_hybrid_equivalence.py
===docs about golden set / licensing===
docs/COMMERCIALIZATION.md
docs/GOLDEN_DATA_SHARING.md
docs/GOLDEN_SET_IMPLEMENTATION.md
docs/GOLDEN_SET_PHASE1_VIDEOS.md
docs/GOLDEN_SET_VENDORING.md
docs/ROORA_LABELING_GUIDELINES.md
docs/archives/COMMERCIAL_READINESS_REVIEW.md
docs/archives/MONETIZATION_AUDIT.md

20. assistant (2026-06-16T02:25:30.466Z)

There's a `docs/GOLDEN\_DATA\_SHARING.md` ? that's directly on point. Let me read it and the golden-set Phase1 videos doc (which should reveal video provenance) plus the trajectory CSV format.

21. user (2026-06-16T02:25:30.948Z)

```
1      # Golden / GPU-Extractor Data Sharing ? GPU box ? dev sessions
2
3      **Status:** convention + plan (2026-06-15). Tracked in **#175**.
4      **Purpose:** a single cloud-synced location for the GPU-extracted golden artifacts
(trajec-tory CSVs, `*_golden_features.json`, manifests, baseline snapshots) so the **GPU box**
and **CPU dev/agent sessions** share the same files ? without committing large data to git.
5
6      ## Why this exists
7      The rally-quality measurement is split across two machines by capability:
8      - **`backend/eval/fusion_golden.py`** (shuttle+person feature **extraction**) needs the
**videos**, the **trajectory CSVs**, the **`.rallies.csv` golden labels**, and a **GPU/WSL**
stack ? only the GPU box can run it.
9      - **`backend/eval/fusion_compare.py`** / **`backend/eval/ablation.py`** (the LOVO
**litmus**: ?F1, bootstrap CI, paired sign test) are **pure-CPU re-scoring** that read only the
small `*_golden_features.json` ? any session (incl. a CPU dev container) can run them.
10
11     Today those artifacts sit in gitignored `output/` (per-machine) and scattered local
folders (`...\Annotation Setup\Collect\Trajectories\`, `...\Golden Labelled\`). Nothing syncs
them between machines, so a CPU session cannot see the GPU box's extraction. This wires that gap
with OneDrive.
12
13     ## Canonical shared folder
14     `%USERPROFILE%\OneDrive\rally-annotator\golden-data\` ?
`C:\Users\avidu\OneDrive\rally-annotator\golden-data\`
15
16     OneDrive is present + signed-in on both the GPU box and the dev machine under the same
account, so it auto-syncs. Layout (date/size-tagged so a re-extraction is a new folder, never an
overwrite):
17
18     ```
19     golden-data/
20     README.md
21     features/<corpus-tag>/      *_golden_features.json      (small ? the key shared
artifact)
22     trajectories/<corpus-tag>/  *_predict.csv / *_wasb.csv (medium ? reusable across
runs)
23     manifests/                  *_lovo_manifest.json
24     baselines/                  leaderboard / regression snapshots (or keep tracked in
```

```

eval_baselines/)
25     ``
26
27     `<corpus-tag>` = e.g. `2026-06-15-n6` (date + corpus size). **Do NOT** put raw videos
here (tens of GB) ? keep those local to the GPU box.
28
29     ## The GPU ? CPU workflow
30     1. **GPU box** (videos + CSVs + labels present):
31         ``
32         python -m backend.eval.fusion_golden --manifest <manifest> \
33             --out-dir "%USERPROFILE%\OneDrive\rally-annotator\golden-
data\features\2026-06-15-n6"
34         ``
35         (or extract to local `output/` then copy the `_golden_features.json` into the shared
folder).
36     2. OneDrive syncs the JSONs to the dev machine automatically (small files, seconds).
37     3. **CPU dev/agent session** points the analysis at the shared folder:
38         ``
39         python -m backend.eval.fusion_compare --features-dir "...golden-
data\features\2026-06-15-n6" --record
40         python -m backend.eval.ablation --features-dir "...golden-
data\features\2026-06-15-n6" --granularity group
41         ``
42         ? real person-feature ?F1 / CI / sign test. Copy the JSONs into the repo's `output/`
to also light up the `skipif` golden-litmus tests
(`tests/test_ablation.py::test_litmus_reproduces_gate_a`, `tests/test_fusion_compare.py`).
43
44     ## Optional code ergonomics (in #175, not required)
45     - A `RALLY_GOLDEN_DIR` env var (default `output/`) honored by `fusion_golden` /
`fusion_compare` / `ablation` so neither machine hardcodes the path. (The CLI `--out-dir` /
`--features-dir` flags already make this a convention; the env var is just ergonomics.)
46     - A tiny `scripts/sync_golden.py` to copy `output/*_golden_features.json` ? the shared
folder.
47
48     ## Guardrails
49     - The shared folder lives **outside the repo** ? never committed. Keep `output/`
gitignored and do **not** symlink the shared folder into a tracked path.
50     - The features derive from the **Proprietary** corpus ? keep the OneDrive folder
**private** (no public share links).
51
52     See also: `docs/NEXT_STEPS.md` (the "Next-session pickup" anchor),
`docs/MULTI_SIGNAL_FUSION_PLAN.md` (P3), `docs/TRACKNET_WSL_SETUP.md` (the GPU/WSL detector
setup).
53

```

22. user (2026-06-16T02:25:31.422Z)

```

1     # Phase-1 Golden-Set Video Collection Spec
2
3     **Status:** Design v1 . **Date:** 2026-06-02
4     **Companion docs:** [GOLDEN_SET_VENDORING.md](GOLDEN_SET_VENDORING.md) (vendor process,
ADR-006) . [VIDEO_RECORDING_GUIDELINES.md](VIDEO_RECORDING_GUIDELINES.md) (capture spec the
user-facing guide derives from) . scoring in [EVALUATION.md](EVALUATION.md).
5
6     What footage to gather for the **first round** of golden-set creation so the trained
rally segmenter reaches good precision/recall ? and so the eval number actually predicts
production. Golden-set creation is **iterative**: expect multiple rounds, expanding toward
whatever axis round-1 metrics show is weakest.
7
8     ---
9
10    ## 1. The principle that dominates everything: match the deployment distribution
11
12    The CV literature is consistent: **models trained on broadcast footage fail to
generalize to self-recorded amateur video** ? broadcast uses a "straight, fixed camera with

```

minimum variability," while real non-elite footage has limited coverage, occlusions, and inconsistent viewpoints ([team-sports survey](https://link.springer.com/article/10.1007/s10462-024-10934-9), [unstructured-video event detection](https://arxiv.org/pdf/2002.08097)). This is the **same failure mode as ADR-004** (broadcast cameras **track players**, so player pixel-motion inverts during rallies).

13
14 Our product ingests **users' own match recordings** (a fixed phone/GoPro on a tripod).
Therefore:

15
16 - The golden set ? **especially the held-out eval split** ? must be dominated by **static, self-recorded, full-court** footage that mirrors real users.
17 - **ShuttleSet / broadcast** footage is fine as **training** augmentation and as a contrast set, but must be **minimal in eval**. Measuring precision/recall on broadcast yields a number that does **not** predict production.

18
19 ---
20
21 ## 2. Primary recording profile (also the basis of the user-facing capture guide)

22
23 Grounded in the badminton-CV setup literature (camera ~2 m behind the baseline, ~4.5 m elevated, ~30° down-tilt, whole court + headroom above the net ?
[source](https://pmc.ncbi.nlm.nih.gov/articles/PMC10747833/)):

24
25 | Setting | Spec | Why |
26 |---|---|---|
27 | Mount | Fixed tripod; **no pan/zoom** during play | Avoids the ADR-004 camera-tracking trap |
28 | Position | Behind one baseline/end, centered | Full-court, symmetric view |
29 | Height | **3-5 m** elevated (~4.5 m ideal) | De-occludes the far player & net |
30 | Tilt | **~30° down** | Optimal coverage, low keystoneing |
31 | Framing | Entire court **+ headroom** above the net; players never leave frame |
Detection + court-relevance need it |
32 | Lighting | Even; no backlight/glare | Reduces blur & false motion |

33
34 Most of round 1 should use this profile; a minority deliberately varies it (§4) to probe robustness.

35
36 ---
37
38 ## 3. Resolution & frame rate (per sport)
39
40 A controlled study: table-tennis ball detection was **22.5% @ 30 fps ? 93.2% @ 60 fps** (120 fps ? 60 fps) ([Nature Sci Rep](https://www.nature.com/articles/s41598-023-42966-6)); OpenTTGames records at 120 fps. The ball/shuttle is the camera-invariant rally signal (our WASB substrate, ADR-001), so fps is **not** a nicety for fast sports.

41
42 | Sport | Min resolution | Recommended fps | Note |
43 |---|---|---|---|
44 | Badminton | 1080p (720p floor) | 30 ok, **50/60 better** | Shuttle decelerates fast; 60 sharpens smashes |
45 | Pickleball | 1080p | **30-60** | Ball slower than TT; 30 likely fine, 60 safer |
46 | Table tennis | 1080p | **60 min** (120 ideal) | 30 fps collapses ball detection |

47
48 > These are **literature-backed starting points**; the published minimums get finalized empirically against our golden set (VIDEO_RECORDING_GUIDELINES.md §3).

49
50 ---
51
52 ## 4. The Phase-1 diversity matrix

53
54 Cover the **high-impact axes** now; don't chase rare conditions in round 1. "Guards" = whether the axis mostly protects **P**recision (not calling dead-time a rally) or **R**ecall (not missing rallies).

55
56 | Dimension | Vary across? | Guards | Round-1 target |

```

57 |---|---|---|---|
58 | **Sport** | badminton, pickleball, table tennis | both | all 3 |
59 | **Format** | singles **and** doubles (badminton, pickleball) | R | both for those 2 |
60 | **Camera profile** | mostly primary-static; 1?2 side-elevated; 1?2 lower/phone-height;
**1?2 broadcast (contrast only)** | both | ~80% static / ~20% variants |
61 | **Venue / court color** | wood, green, blue synthetic | P | ?3 venues |
62 | **Lighting** | bright gym, dim club, mixed/backlit | both | ?3 conditions |
63 | **Background clutter** | empty hall vs people moving behind court | **P** (crowd
motion ? false positives) | both |
64 | **Players** | skill, attire colors, body types, L/R-handed, skin tone/age | R |
spread; don't reuse one pair |
65 | **Rally edge cases** | service faults, lets/replays, 1?2-shot & long rallies, net-
cords, coaching pauses, warm-up | boundary **P/R** | ensure present |
66 | **Resolution/fps ablation** | same match re-encoded 480/720/1080p; 30 vs 60 fps |
calibration | 1 matched set/sport |
67
68 ---
69
70 ## 5. Split it so the precision/recall number is honest (no train/eval leakage)
71
72 The #1 video-ML mistake: the **same match in train and eval inflates the score** ([data-
leakage guide](https://towardsdatascience.com/data-science-mistakes-to-avoid-data-
leakage-e447f88aae1c/)).
73
74 - **Split by match / recording session ? never by clip.** All clips from one match go
entirely to train **or** eval.
75 - **Hold out whole *conditions*** (a venue, a camera profile, a player pair the model
never saw) so eval measures generalization, not memorization.
76 - Reserve **~? of each sport as a deployment-like held-out eval set** (static, self-
recorded). Broadcast/ShuttleSet may enter train but should be **minimal in eval**.
77
78 > **Guardrails ? TODO (design with the user):** train and eval live in **separate
catalogs** today, kept apart by convention. We want hard guardrails ? permissions / explicit
approval / re-confirmation at ingest & export ? so a video can **never** silently land in both.
Tracked in BACKLOG.md; to be designed in a later session.
79
80 ---
81
82 ## 6. Concrete "round 1" shopping list (per sport)
83
84 Aligns with the 12?15-videos/sport scale plan; the 3-clip Roora calibration is a subset
of this.
85
86 - **8 static-primary** clips spanning venues / lighting / skill (?4 singles, 4 doubles
where applicable)
87 - **2 camera-variant** clips (side-elevated, lower height) ? robustness probe
88 - **1 resolution/fps ablation set** (one match @ 480/720/1080p, 30 & 60 fps)
89 - **1 broadcast contrast** clip (ShuttleSet) ? **train/contrast only**
90 - Each ~2?4 min, **deliberately including the edge cases** above ? **~12 clips/sport**;
hold out **4 whole matches** for eval.
91
92 ---
93
94 ## 7. Deliberately defer to later rounds
95
96 Outdoor courts, phone-handheld/jitter, extreme angles, 4K, unusual net/court variants,
crowd-heavy tournaments. Add these once round-1 precision/recall shows the weakest axis, then
expand toward it (find the quality "knee" ? VIDEO_RECORDING_GUIDELINES.md S3).
97
98 ---
99
100 ## 8. Two cautions specific to precision (the stated goal)
101
102 - **Backgrounds with people moving** behind the court are the biggest false-positive
driver. Include some so the model learns to reject them ? and make sure the annotation

```


guideline's "ignore all dead time" rule is airtight.

```

103 - **Warm-up / knock-up / practice swings** look like rallies. Capture some, and confirm
the vendor **excludes** them (the Roora guide now has an explicit rule + a how-to-spot-it tip).
Early precision is bounded more by **guideline / inter-annotator consistency (IAA)** than by raw
data volume.
104
105 ---
106
107 ## 9. Sources
108
109 - [Team-sports action-recognition survey ? Springer
2024](https://link.springer.com/article/10.1007/s10462-024-10934-9)
110 - [Unstructured-video event detection ? arXiv
2002.08097](https://arxiv.org/pdf/2002.08097)
111 - [Badminton service-height CV setup (camera 4.5 m / 30°) ?
PMC](https://pmc.ncbi.nlm.nih.gov/articles/PMC10747833/)
112 - [VideoBadminton dataset ? arXiv 2403.12385](https://arxiv.org/html/2403.12385v1)
113 - [Table-tennis frame-rate vs detection accuracy ? Nature Sci
Rep](https://www.nature.com/articles/s41598-023-42966-6)
114 - [OpenTTGames (120 fps) dataset](https://lab.osai.ai/)
115 - [Action-spotting / segmentation survey ? arXiv
2505.03991](https://arxiv.org/html/2505.03991v2)
116 - [Data-leakage in train/test splits](https://towardsdatascience.com/data-science-
mistakes-to-avoid-data-leakage-e447f88aae1c/)
117

```

23. assistant (2026-06-16T02:25:36.070Z)

This is very informative. `GOLDEN\_DATA\_SHARING.md` explicitly states the golden corpus is **Proprietary** and the derived features must NOT be committed (kept in private OneDrive, repo's `output/` gitignored). Let me confirm the trajectory CSV / golden_features format, check the licensing/privacy guardrail docs, and verify the video provenance (self-recorded vs broadcast) plus minors handling.

24. user (2026-06-16T02:25:37.491Z)

```

1  """Offline golden-set person-feature extractor ? the fusion-P3 measurement's GPU step.
2
3  Mirrors `distill_local`'s CSV-driven flow but ADDS the ~5 person-cue features per
shuttle
4  candidate window, by running `PersonDetector` over the ORIGINAL video. It reuses the
5  EXISTING golden trajectory CSVs (so **no WASB re-run**), needs **no Gemini and no DB**,
and
6  writes one small JSON per video. The LOVO comparison (`fusion_compare.py`) then runs
100% on
7  CPU offline. So the GPU touches exactly one thing ? person detection ? and the lift is
8  measured by the shipped experiment harness; person features enter only as feature
columns the
9  learned scorer weights (no special-casing, default-OFF until the golden ?F1 justifies
them).
10
11  Run (on the GPU box):
12      python -m backend.eval.fusion_golden --manifest output/human_lovo_manifest.json
--out-dir output
13
14  Each output `output/<name>_golden_features.json` is the replayable feature substrate ?
re-run
15  `fusion_compare` with different variants/thresholds forever without re-detecting on the
GPU.
16  """
17  from __future__ import annotations
18
19  import argparse
20  import json
21  import os

```

```

22     import time
23     from typing import Any, Dict, List, Optional
24
25     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
26     from backend.pipeline.detectors import rally_gate
27     from backend.pipeline.detectors import fusion_features as ff
28     from backend.eval import distill_local
29     from backend.eval.calibrate_wasb import load_golden_gts
30     from backend.eval.training import label_candidates
31     from backend.utils.run_telemetry import RunTelemetry
32
33     # The 5 fps/resolution-invariant shuttle features distill_local already computes.
34     SHUTTLE_FEATURE_NAMES = list(distill_local.FEATURE_NAMES)
35     # The 5 person-cue features (fusion_features.PERSON_FEATURE_NAMES).
36     PERSON_FEATURE_NAMES = list(ff.PERSON_FEATURE_NAMES)
37
38
39     def _derive_video_path(spec: Dict[str, Any]) -> str:
40         """Original MP4 for a manifest entry ? explicit `video_path`, else derived from the
41         golden labels path (`X.rallies.csv` -> `X.MP4`, the golden-set naming
42         convention)."""
43         if spec.get("video_path"):
44             return str(spec["video_path"])
45         labels = str(spec["labels"])
46         if labels.endswith(".rallies.csv"):
47             return labels[: -len(".rallies.csv")] + ".MP4"
48         raise ValueError(f"{spec.get('name')}: no video_path and labels {labels!r} isn't
49         *.rallies.csv")
50
51     def extract_video(spec: Dict[str, Any], detector: Optional[Any] = None,
52                      frame_skip: int = 3, iou: float = 0.5,
53                      log_every_sec: float = 15.0,
54                      telemetry: Optional[Any] = None) -> Dict[str, Any]:
55         """One golden video -> a feature record (shuttle + person features per candidate
56         window).
57         Returns `{name, fps, frame_width,
58         candidates:[{start,end,shuttle{ },person{ },label}], gts}`.
59         `detector` (anything with `detections_over_windows`) is injectable for GPU-free
60         tests;
61         when None a real `PersonDetector` is built (loads torchvision on first use).
62         `telemetry` (a `RunTelemetry` or None) records per-frame velocity + machine-health
63         snapshots
64         from the detection progress callback so a hard kill leaves a forensic runlog.
65         """
66         fps, fw = float(spec["fps"]), float(spec["frame_width"])
67         traj = str(spec["trajectory"])
68         points = TrackNetRunner.parse_trajectory_csv(traj)
69         intervals, x_shuttle = distill_local.build_candidates(traj, fps, fw)
70         gts = load_golden_gts(str(spec["labels"]))
71         labels = label_candidates(intervals, gts, iou)
72         # Net axis for the person two-sided split ? reuse rally_gate's camera-agnostic
73         estimate
74         # (parity with the shuttle net-crossing gate) rather than hard-coding an
75         orientation.
76         axis, net_px, _span = rally_gate.estimate_play_axis(points)
77         video_path = _derive_video_path(spec)
78
79         if detector is None:
80             from backend.pipeline.detectors.person_detector import PersonDetector
81             detector = PersonDetector({"indexing": {"use_gpu": True}})
82
83         # Single-pass person detection over ALL candidate windows (open the video ONCE, read
84         # forward ? no per-window seek). Then compute features per window (pure, no GPU).

```

```

79         n = len(intervals)
80         win_frames = [(round(s * fps), round(e * fps)) for (s, e) in intervals]
81         print(f"[fusion_golden]   {spec['name']}: {n} windows ? person-detecting in one
pass...", flush=True)
82         t0 = time.monotonic()
83         state = {"last": t0}
84
85         def _progress(done: int, total: int) -> None:
86             if telemetry is not None:
87                 telemetry.snapshot(done, total, phase=spec["name"])
88             now = time.monotonic()
89             if now - state["last"] >= log_every_sec or done >= total:
90                 el = now - t0

```

25. user (2026-06-16T02:25:39.997Z)

===GOLDEN_SET_VENDORING provenance/licensing===

4: **Owner decisions captured:** licensed/owned video only · rally `[start,end)` depth · tiered turnaround (fast automated + slow human) · vendor bake-off · **Bangalore/India-local vendor preference**

26: ## 2. Why this also solves the privacy nuance

30: - **Golden sets are built only from footage we license or own** ? starting with the founder's **constantly-growing personal collection** of match videos, plus ShuttleSet (ADR-003) and any purpose-recorded clips. **No end-user uploads ever flow to a vendor.**

34: > **Honest tradeoff:** a licensed/owned corpus may not perfectly mirror real users' cameras, courts, and lighting. A green regression on our corpus is therefore **strong evidence**, not a **guarantee**, for production traffic. Revisit if field behavior diverges from corpus behavior ? and see VIDEO_RECORDING_GUIDELINES.md, which aims to shrink that gap by steering users toward capture conditions our corpus represents.

44: - **Edge cases:** lets/replays, service faults, between-point walking, coaching pauses, mid-rally net-cord, broadcast replays and graphics.

51: One trusted expert (initially: the founder) hand-annotates **5?10 licensed clips** to near-perfect. This is the answer key vendors are graded against ? kept private during the bake-off.

188: 2. **Corpus representativeness** ? how well does the owned/licensed collection mirror eventual user capture? (Ties directly to VIDEO_RECORDING_GUIDELINES.md.)

26. user (2026-06-16T02:25:41.972Z)

===minors / consent / privacy across docs===

docs/ABLATION_LAYER.md:4: `backend/eval/ablation.py` runs it and it EXCLUDES one signal, measures whether quality dropped,

docs/ANALYZERS_AND_RECONCILERS.md:249: column*****; the scorer ***learns*** to down-weight windows inside the span. **Warm-up is the poster child** for

docs/ANALYZERS_AND_RECONCILERS.md:283: distilled labels or a new (minors-gated) annotation task ? the same gate as feature-level fusion. If you can

docs/ANALYZERS_AND_RECONCILERS.md:521: - **R3** ? corrections persistence + opt-in apply*****: JSON sidecar first, then the `corrections` table

docs/CODE_MAP.md:34: | A throwaway repro / debug script | `scratch/` | **Gitignored** + excluded from ruff/pytest. Never imported by app code. |

docs/COMMERCIALIZATION.md:43: | C14 | **Owned-model base licensing + training-data policy** | **P1** | ? ? ? | Owned video-QA model must build on a **commercial-clean (MIT/Apache-2.0/BSD)**, video-capable***** base ? primary **Qwen3-VL (Apache-2.0)**, fallback **InternVL3 (MIT)**. **Gemma 4 = AMBER / bench-only** (Apache grant likely but Prohibited-Use posture unconfirmed ? counsel needed; not a build target yet). **Reject Gemma 3** (viral derivative clause ? permanent Google PUP), Llama Vision (naming + 700M-MAU), `gpt-oss` (text-only). **Hard rule: never train on paid-LLM (Gemini/OpenAI/Anthropic) outputs** (competing-model clause). The moat is the **consented dataset + eval harness**, not the model. | **MIT/Apache-2.0/BSD** for code+data+weights; **open-model teachers only**; separate explicit **training opt-in**; **exclude minors**; per-clip consent ledger. See **OWNED_MODEL_IMPLEMENTATION_PLAN.md** (authoritative). |

docs/FIELD_VISIT_PLAYBOOK.md:62: students are 18 or older ? "senior batch" players who are minors do NOT count. Then ask 2?3 of them:

docs/FIELD_VISIT_PLAYBOOK.md:106: the best possible signal: write it down verbatim, say **"I'll pass that straight to Avi"**, and promise nothing. **Hard rules:** written consent from the academy

docs/FIELD_VISIT_PLAYBOOK.md:107: AND the adult players being recorded; **sessions with minors**

are not recorded for our library** ? adult sessions only;

docs/GOLDEN_SET_PHASE1_VIDEOS.md:103:- **Warm-up / knock-up / practice swings** look like rallies. Capture some, and confirm the vendor **excludes** them (the Roora guide now has an explicit rule + a how-to-spot-it tip). Early precision is bounded more by **guideline / inter-annotator consistency (IAA)** than by raw data volume.

docs/GOLDEN_SET_VENDORING.md:157:- **Karya (Bengaluru)** ? confirmed local and reputable, but **speech/text/translation only, no video** ? **excluded** for our use case.

docs/I18N_PLAN.md:226:- **No minors / biometrics scope creep.** i18n touches presentation only; it introduces no new

docs/KHELSUTRA_PRODUCT_OVERVIEW.md:97:tool without your **clear, explicit agreement** ? it is opt-in, never automatic. **Footage involving

docs/KHELSUTRA_PRODUCT_OVERVIEW.md:98:minors is never used to improve the tool and is never published.** We never publish your footage; you

docs/KHELSUTRA_PRODUCT_OVERVIEW.md:103:to improve the tool without your opt-in.

docs/KHELSUTRA_PRODUCT_OVERVIEW.md:177:your explicit opt-in. These are large videos that need real processing, so your video is handled securely

docs/KHELSUTRA_PRODUCT_OVERVIEW.md:178:on our infrastructure to make your reel. **Footage involving minors is never used to improve the tool and

docs/MULTI_SIGNAL_FUSION_PLAN.md:203: and the **paid-LLM/training guardrail + minors-exclusion** apply to any future person-training data.

docs/NEXT_STEPS.md:62:- **Key finding (no re-investigation needed):** the owner's "held-shuttle counted as a rally" bug is **NOT a missing capability** ?

`backend/pipeline/detectors/rally\_gate.py` already implements the net-crossing ***"Gate A"*** (`apply\_gate(..., min\_crossings=2)`) that kills service-feed/held-shuttle windows, but it is **only called from eval drivers** (and there was no `min\_crossings` knob in `IndexingConfig`).

Wiring Gate A into the live segmenter (now via fusion_hybrid) was the single cheapest, highest-confidence quality win. (P2 FusionSegmenter specifics: registry `fusion\_hybrid`, **default-OFF**, seeds from substrate, persists `net\_crossings` + optional person features, optional served Gate A/B via config knobs; adversarially reviewed; suite green. See `tests/test\_served\_gate\_a\_regression.py` for the honest finding that on production windowing Gate A is neutral/negative in some cases, hence kept opt-in with knob for safety, and the leaderboard revert note.)

docs/NEXT_STEPS.md:65: 1. (P2 first-step, largely done via fusion_hybrid) Wire Gate A into production served path + config knob (min_crossings); apply in hybrids at runtime. (Now available opt-in; served neutral on production windowing per litmus ? kept opt-in with knob for safety.)

docs/NEXT_STEPS.md:78: `VIDEO\_RECORDING\_GUIDELINES.md`; **adult sessions only for the training pool** (consent guardrail). Per video:

docs/NEXT_STEPS.md:93: (consent at capture; demo served via the Gemini path or human-polish ? the reel pattern exists:

docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md:21:- **The moat = the consented dataset + the eval harness, not the weights.**

docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md:29:- **Exclude minors; per-user training data needs a separate explicit opt-in; split data by match.**

docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md:104:shot_count, label_type, source(human|pseudo|vendor), consent/training_opt_in, split(BY MATCH), trajectory_ref,

docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md:165:VRAM/token cost · single-model-multisport-temporal generalization unproven · DPA + no-train (minors-excluded) for cloud

docs/PERSONALIZATION_PLAN.md:52: detection path** (today it lives in the opt-in

`FusionSegmenter`, which is not the default

docs/PERSONALIZATION_PLAN.md:56: **free-tier videos ARE pooled** into the baseline (consent baked into the free terms;

docs/PERSONALIZATION_PLAN.md:57: minors-exclusion still enforced) ? the clean value-exchange that feeds the owner-trained base model.

docs/PERSONALIZATION_PLAN.md:89: MINOR-vs-MAJOR upgrade switch.

docs/PERSONALIZATION_PLAN.md:90:- **v6** ? *pending.* `contribution\_consent` ledger (default-deny; minors-exclusion enforced **at

docs/PERSONALIZATION_PLAN.md:100:| 4 | Contribution flywheel (opt-in data/video/diagnostics ? shared baseline; payoff ? contribution; consent + minors-exclusion) | Phase 2 |

docs/PERSONALIZATION_PLAN.md:122:writer); recall-protecting de-bias prior; v6 consent ledger + minors-exclusion; **opt-in contribution

docs/PERSONALIZATION_PLAN.md:127:upgrade governance (MINOR carry / MAJOR re-fit on

`feature\_schema\_hash`) + optional federated nudge

docs/PERSONALIZATION_PLAN.md:137:4. **Local-install flywheel default** ? fully-local zero-pooling until consent is explicitly granted?

docs/PERSONALIZATION_PLAN.md:146: near-term pooling carries the full cloud (DPA / consent / minors) posture.

docs/PLATFORM_ARCHITECTURE.md:262: fine-tuned on sports video (including **consented, user-contributed** footage) ? that

docs/PLATFORM_ARCHITECTURE.md:269:> assets are **(a) the consented, badminton-specific video?QA dataset** and **(b) the

docs/PLATFORM_ARCHITECTURE.md:300:(consented user vid (fine-tune an open, (extend the existing (self-host inference

docs/PLATFORM_ARCHITECTURE.md:337: sustained video-QA volume **** a defensible consented dataset **** real per-call API

27. assistant (2026-06-16T02:25:48.980Z)

I now have comprehensive coverage. Let me verify the golden_features JSON shape (the actual per-video derived stream) and the trajectory CSV format/size to nail the size budget, and confirm the corpus size (n=6).

28. user (2026-06-16T02:25:50.115Z)

```

90         el = now - t0
91         rate = done / el if el > 0 else 0.0
92         eta = (total - done) / rate if rate > 0 else 0.0
93         pct = 100 * done // total if total else 0
94         print(f"[fusion_golden]   {spec['name']}: detect frame {done}/{total}
({pct}%) "
95             f"| {el:.0f}s elapsed | ETA {eta:.0f}s", flush=True)
96         state["last"] = now
97
98     per_window = detector.detections_over_windows(video_path, win_frames, frame_skip,
99                                                  progress_cb=_progress)
100
101     candidates: List[Dict[str, Any]] = []
102     for i, ((s, e), xs) in enumerate(zip(intervals, x_shuttle)):
103         det = per_window[i]
104         det_fw = det.get("frame_width") or fw
105         det_fh = det.get("frame_height") or 0.0
106         # Normalise the (pixel) net coord to 0-1 on the play axis, matching the foot-
point
107         # normalisation fusion_features uses (x/width, y/height).
108         if axis == "x":
109             net_norm = net_px / det_fw if det_fw > 0 else 0.5
110         else:
111             net_norm = net_px / det_fh if det_fh > 0 else 0.5
112         sf, ef = win_frames[i]
113         shuttle_in = [p for p in points if sf <= p.frame <= ef]
114         person = ff.compute_person_features(det["frames"], shuttle_in, det_fw, det_fh,
115                                           court_polygon=None, net=(axis, net_norm))
116         candidates.append({
117             "start": float(s), "end": float(e),
118             "shuttle": {k: float(v) for k, v in zip(SHUTTLE_FEATURE_NAMES, xs)},
119             "person": {k: float(v) for k, v in person.items()},
120             "label": int(labels[i]),
121         })
122     return {
123         "name": spec["name"], "fps": fps, "frame_width": fw,
124         "candidates": candidates,
125         "gts": [[float(a), float(b)] for a, b in gts],
126     }
127
128
129 def run(manifest_path: str, out_dir: str, frame_skip: int = 3, iou: float = 0.5,
130        fresh: bool = False, smallest_first: bool = False) -> List[str]:
131     with open(manifest_path, encoding="utf-8") as f:
132         specs = json.load(f)
133     if smallest_first:

```

```

134         # Process small videos first for quick feedback (the slow full match lands
last);
135         # the LOVO result is order-independent. Proxy size by the trajectory CSV (one
row/frame).
136         specs = sorted(specs, key=lambda s: (os.path.getsize(s["trajectory"])
137                                             if os.path.isfile(s.get("trajectory", ""))
else 0))
138         os.makedirs(out_dir, exist_ok=True)
139         # Run-telemetry black box: a long GPU run can be OS-killed with no traceback, so
keep a tiny
140         # rotating runlog of velocity + machine health for the post-mortem. Constructing it
must never
141         # break extraction, so guard the build and fall back to no telemetry on any error.
142         telem: Optional[RunTelemetry]
143         try:
144             telem = RunTelemetry(os.path.join(out_dir, "run-telemetry.jsonl"),
label="fusion_golden")
145         except Exception: # noqa: BLE001 - telemetry wiring must never break the run it
observes
146             telem = None
147         written: List[str] = []
148         for vi, spec in enumerate(specs, 1):
149             out = os.path.join(out_dir, f"{spec['name']}_golden_features.json")
150             # Resumable: skip a video whose JSON already exists (a re-run picks up where a
prior
151             # run left off instead of re-detecting). --fresh forces a redo.
152             if not fresh and os.path.isfile(out):
153                 print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: already
extracted "
154                       f"-> {out} (use --fresh to redo)", flush=True)
155                 written.append(out)
156                 continue
157             print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: extracting...",
flush=True)
158             t0 = time.monotonic()
159             if telem is not None:

```

29. user (2026-06-16T02:25:51.762Z)

```

===trajectory CSV parse format===
7:     stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
9:     3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
18:import csv
42:     keep_frames: bool = False # retain staged video after success
(debug only)
58:     keep_frames=bool(tn.get("keep_frames", False)),
81:     frame: int
88:     """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.
122:     """Run predict.py on a video. Returns the Windows path to the produced CSV, or None.
125:     WSL writes into via its /mnt view, so the Windows process can read the CSV
127:     therefore predict.py's ``<stem>_predict.csv`` output ? so two videos that share
128:     a filename cannot collide on the output CSV.
148:     # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
154:     # Namespace by video_id so two same-filename videos don't collide on the CSV.
156:     # under the namespaced name propagates into "<key>_predict.csv".
158:     expected_csv_win = os.path.join(output_win_dir, f"{key}_predict.csv")
167:     # No staging: predict.py reads /mnt directly and writes "<stem>_predict.csv".
170:     expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")
199:     if not os.path.exists(expected_csv_win):
200:         logger.error("TrackNet finished but expected CSV not found at %s",
expected_csv_win)
202:         logger.info("TrackNet trajectory CSV: %s", expected_csv_win)
204:         # Success ? the CSV is the durable output; drop the staged video copy (a pure,
206:         if self.cfg.stage_in_wsl and not self.cfg.keep_frames:
208:             "(set indexing.tracknet.keep_frames=true to retain).", key)

```

```
210:         return expected_csv_win
234:     # ----- CSV -> windows
```

30. assistant (2026-06-16T02:26:03.481Z)

I have everything I need. The trajectory CSV is `Frame,Visibility,X,Y` (one row per frame). Let me do a final quick calc on size budget ? confirm corpus size and typical video lengths.

The corpus is n=6 (from `heuristic\_lovo\_n6.json` and the LOVO docs). Trajectory CSV is one row per frame: `Frame,Visibility,X,Y` (~4 small ints/floats, ~20-25 bytes/row). A 2-4 min clip at 30-60fps = ~3,600-14,400 rows ? roughly 80KB-350KB per trajectory CSV. The `\_golden\_features.json` is much smaller (one record per candidate window, ~10 features each, tens of windows per video ? a few KB to tens of KB).

I now have a complete picture. Let me compile the structured answer.

31. user (2026-06-16T02:27:22.695Z)

Structured output provided successfully

32. assistant (2026-06-16T02:27:36.145Z)

I've completed the investigation and returned the structured findings. Summary of what I determined:

****Where fixtures belong / what's gitignored:**** Tests go in `tests/test\_\*.py` (pure/mock, 70% cov floor); tracked metric baselines go in `eval\_baselines/\*.json`; bulk derived data goes to gitignored `output/`. `.gitignore` covers `output/`, `scratch/`, `\*.db`, `\*.pt`, `.env`. The repo commits NO raw/derived data fixtures today ? zero tracked CSV/JSONL, no test data dirs; the only structured-data precedent is tiny aggregate-metric JSON in `eval\_baselines/`.

****git-lfs:**** Installed (v3.5.1, global filters) but NOT in use ? no `.gitattributes` anywhere. The repo chose "keep big data out of git" (gitignored `output/` + private OneDrive) over LFS.

****Schema/version conventions:**** `PRAGMA user\_version` migration ladder (currently v5) in `database.py`; detector-agnostic schema (common fields as columns + `stats` JSON keyed by a `detector` string); `eval\_baselines` JSON carries provenance (`computed` repro command). A fixture should mirror these ? version tag + provenance + column-vs-stats split.

****Licensing/privacy (the load-bearing answer):**** Committing derived numeric streams is *technically* clean ? videos are owner/licensed self-recorded + ShuttleSet (never end-user uploads or broadcast scraping), streams are abstract numbers not frames, and they come from MIT detectors not paid-LLM output. BUT `docs/GOLDEN\_DATA\_SHARING.md` explicitly designates the corpus ****Proprietary**** and mandates its derived features live in ****private OneDrive, never committed****. So committing even derived data reverses a stated convention and is an owner-gated product/privacy decision. To stay clean: commit only numeric streams (no frames), only from owned + minor-free-at-source videos, with explicit per-video provenance/consent tags.

****Size budget:**** n=6 golden videos. Per-video trajectory CSV (`Frame,Visibility,X,Y`, one row/frame) ? 80KB-350KB; full n=6 ? <2MB; feature JSONs are smaller still. Comfortably git-sized ? LFS not needed.

One flag: `git pull --ff-only` failed ("Cannot fast-forward to multiple branches"), so the local checkout may lag origin.